

Dossier d'Architecture Technique (DAT) : Smart Grid

1. Vision Globale du Design Architectural : L'Architecture Hybride Lambda-Kappa

Pour faire cohabiter l'analytique temps réel à l'échelle de la seconde et le traitement batch massif sur des historiques profonds, l'architecture repose sur un modèle hybride.

- **Le Speed Layer (Temps réel / Stream)** : Capture la télémétrie des compteurs connectés à la seconde, calcule des métriques glissantes, détecte les anomalies immédiates (surtensions, ruptures) et alimente le modèle en production pour des décisions instantanées.
- **Le Batch Layer (Traitements par lots)** : Récupère les données brutes archivées, réconcilie les données d'ingestion avec les sources externes (API météo, données de marché historiques), recalcule des agrégats parfaits et réentraîne les modèles prédictifs.
- **Le Serving Layer (Couche d'exposition)** : Centralise les données finalisées pour la consommation applicative et décisionnelle.

2. Découpage de la Stack Technique et Rôles Précis

A. Ingestion et Routage des Flux (Kafka & RabbitMQ)

- **RabbitMQ (Edge/Ingestion de proximité)** : Utilisé comme tampon de premier niveau pour collecter les messages légers (MQTT/AMQP) provenant des concentrateurs régionaux ou des passerelles de compteurs connectés. Il gère efficacement les pics de connexion intermittents et applique un pré-routage simple.
- **Apache Kafka (Event Backbone central)** : RabbitMQ transfère les messages validés vers Kafka (via un connecteur Kafka Connect). Kafka sert de journal de commits distribué et immuable.
 - *Topics clés* : `telemetry.raw.smartmeters`, `grid.alerts`, `market.prices.realtime`.
 - *Configuration* : Facteur de réplication de 3, partitionnement basé sur l'identifiant du nœud de distribution électrique (`grid_node_id`) pour garantir l'ordre des messages par zone géographique.

B. Traitement et Consommation (Flink & Spark)

- **Apache Flink (Stream Processing)** : Consomme en continu les topics Kafka. Il applique des fonctions de fenêtrage temporel (*Tumbling* et *Sliding Windows*) pour

calculer des agrégats à la volée (ex: consommation moyenne par nœud toutes les 5 minutes) et pousse les alertes de surcharge vers Redis.

- **Apache Spark (Batch Processing)** : Exécuté de manière planifiée pour traiter les volumes massifs. Spark SQL et l'API Dataframe lisent la couche *Bronze* du Data Lake pour nettoyer les données, traiter les valeurs manquantes (imputation par interpolation linéaire ou moyenne historique) et consolider les couches *Silver* et *Gold*.

C. Stockage Multi-Modèles et Architecture en Médaillon (MinIO, PostgreSQL, MongoDB, Redis)

Le stockage est orchestré selon le pattern du Data Lakehouse, matérialisé par des compartiments sur un stockage objet distribué compatible S3 (**MinIO**), structuré en trois couches :

1. **Couche Bronze (Raw)** : Stockage brut immuable des événements Kafka au format JSON ou Avro, partitionné par date (*year=YYYY/month=MM/day=DD/*).
 2. **Couche Silver (Cleaned & Enriched)** : Données nettoyées, dédoublonnées et enrichies (coordonnées géographiques des nœuds ajoutées, fusion avec les données météo réelles). Stockées au format Parquet ou Delta Lake pour permettre des requêtes performantes.
 3. **Couche Gold (Aggregated/Business)** : KPI métiers calculés par Spark et Flink (consommation horaire, prévisions de charge calculées, écarts prévision/réel).
- **Redis (Feature Store & Caching)** : Stocke l'état le plus récent de chaque nœud du réseau et les variables caractéristiques (*features*) calculées en temps réel par Flink. Le modèle de Machine Learning vient interroger Redis à très faible latence (< 5ms) pour récupérer le contexte d'un nœud lors d'une prédiction.
 - **PostgreSQL (Relational Serving Store)** : Base de données cible pour la couche Gold. Elle contient les tables d'administration, la topologie du réseau, l'historique des prévisions validées et sert de source principale pour **Power BI**.
 - **MongoDB (Document Store)** : Reçoit les logs semi-structurés, les métadonnées changeantes des équipements du réseau et les rapports d'intervention technique qui ne s'intègrent pas dans un schéma relationnel strict.

D. Orchestration (Apache Airflow)

Airflow ne touche pas au flux de données temps réel. Il orchestre la branche Batch :

- Déclenchement quotidien du téléchargement des données de réanalyse météo (Open-Meteo API).
- Lancement des applications Spark de consolidation et de nettoyage.
- Planification hebdomadaire ou pilotée par les événements du pipeline de réentraînement du modèle de Machine Learning.
- Vérification de la qualité des données (Data Quality Gates) avant le passage de Silver à Gold.

E. MLOps et cycle de vie des modèles

- **Entraînement** : Un modèle (ex: XGBoost ou un modèle temporel de type LSTM sous framework Python comme PyTorch ou Scikit-Learn) est entraîné pour prédire la charge électrique du réseau à J+1 pour chaque nœud.
- **Inférence** : Un microservice d'inférence (FastAPI conteneurisé) est déployé. Il reçoit les requêtes de décision automatisée, récupère les métriques récentes dans Redis, et retourne la prédiction.
- **Re-entraînement piloté par la donnée (*Data-driven retraining*)** : Airflow compare chaque jour les prévisions du modèle avec la réalité stockée dans PostgreSQL. Si le score de performance (ex: l'erreur absolue moyenne en pourcentage - MAPE) dépasse un seuil de 5% sur une région géographique, Airflow déclenche automatiquement un pipeline de réentraînement du modèle sur les données fraîches de la couche Gold.

F. DevOps et Infrastructure (Terraform, Docker, Kubernetes)

- **Reproductibilité locale et Cloud : Terraform** décrit l'intégralité de l'infrastructure. Il provisionne les clusters **Kubernetes** (AKS, EKS ou une topologie locale type Kind/Minikube), ainsi que les rôles IAM et les compartiments de stockage sur MinIO.
- **Conteneurisation** : Chaque brique (les workers Spark, les gestionnaires de tâches Flink, les DAGs Airflow, le service d'inférence ML) possède son propre fichier **Dockerfile**, assurant des environnements isolés et reproductibles. Les variables d'environnement gèrent les chaînes de connexion pour s'assurer que les bases de données (PostgreSQL, MongoDB) et les environnements d'analyse (Jupyter pour la recherche R&D) communiquent de manière fluide au sein de réseaux virtuels isolés.

G. Observabilité et Restitution (Grafana & Power BI)

- **Grafana** : Collecte les métriques techniques du système via Prometheus (taux d'occupation des files d'attente Kafka, latence des checkpoints Flink, consommation CPU/RAM des pods Kubernetes).
- **Power BI** : Se connecte à PostgreSQL (Couche Gold) pour exposer les rapports métiers aux décideurs. C'est ici que s'illustre la *médiation de la donnée* : au lieu d'afficher une simple courbe brute de prédiction, le tableau de bord intègre des modules visuels expliquant les facteurs clés de la décision de l'algorithme (ex: l'impact de la baisse soudaine des températures ou de la chute de production éolienne sur la marge de sécurité du réseau).

3. Patterns de Design Clés à Déployer

1. **Event Sourcing** : La couche Bronze conserve l'état brut de tous les événements du réseau. Si un bug est détecté dans le code de nettoyage ou dans un modèle, il suffit de "rejouer" le flux d'événements depuis le début de l'histoire pour reconstruire un état parfait.
2. **CQRS (Command Query Responsibility Segregation)** : L'écriture des données à haute fréquence se fait uniquement dans Kafka/MinIO (Commandes), tandis que la lecture analytique s'effectue sur PostgreSQL et Power BI (Requêtes), évitant tout verrouillage de base de données.

3. **Circuit Breaker (Sécurité MLOps)** : Si le service d'inférence en temps réel subit une latence réseau ou si le modèle plante, un pattern Circuit Breaker bascule automatiquement vers une règle métier dégradée (ex: appliquer la consommation de la même journée de la semaine précédente) pour ne jamais bloquer la boucle de décision.